ASSIGNMENT 5 – PART B

B1 – ADMINISTER THE LINUX SERVER

Implementation and Experimental Investigation

Task weight: 7 Marks **Bloom's Level:** K5 – Evaluate

This report documents the implementation plan, completed shell-script logic, Linux administration configuration, validation commands and evaluation for Task B1. System measurements that depend on the student's own Linux machine are intentionally left for execution-time evidence rather than invented.

1. Objective

The objective is to observe a Linux server, automate log and disk monitoring, run a sample portal as a systemd service behind Nginx, restrict inbound network access to SSH/HTTP/HTTPS, and compare the implementation with the Part A design.

2. B1.1 – Observe the System

The following commands provide a baseline view of process activity, memory, virtual memory behaviour, storage devices, filesystems and network sockets.

```
ps -eo pid,ppid,user,stat,%cpu,%mem,comm --sort=-%cpu | head -n 16
top
free -h
vmstat 1 5
iostat -xz 1 2
df -hT
ss -tulpn
```

Interpretation of the measurements

Command	What it measures	Interpretation
ps / top	Processes, parent IDs, state, CPU and memory	Identify resource-heavy processes and process states.
free -h	RAM and swap totals, used and available memory	Available memory is a better indicator of headroom than free memory alone.
vmstat	Runnable/blocked processes, memory, swap, CPU and I/O wait	Sustained swap-in/out or high I/O wait can indicate pressure.
iostat -xz	Device throughput, queueing, await and utilisation	High latency or sustained saturation warrants disk investigation.
df -hT	Filesystem capacity, use percentage and mount point	Find filesystems approaching the alert threshold.

Command	What it measures	Interpretation
ss -tulpn	Listening TCP/UDP sockets and owning processes	Confirm only intended public listeners are exposed.

The included collector script executes these commands and saves a timestamped text record. `iostat` is supplied by the `sysstat` package. Some socket-process details may require elevated privileges. The first vmstat report can represent averages since boot; subsequent lines are interval samples.

Measurement evidence must be collected on the actual Linux VM/server used for the assignment. CPU, RAM, process counts, filesystem use and socket listings vary by machine and time, so this report does not substitute sample values for measured results.

3. B1.2 – Automate Log Analysis

The completed assignment script reads the provided seven-line sample log. In that simplified format, field 7 is the HTTP status code and field 8 is the response size. `awk` extracts the required field, `sort` groups equal values, and `uniq -c` counts each group. The busiest client is found by counting field 1 and sorting the counts in descending order.

Completed TODO logic

```
awk '{print $7}' "$log" | sort | uniq -c | awk '{print $2, $1}' | sort -n -k1,1
awk '{print $1}' "$log" | sort | uniq -c | sort -nr | head -n 1 | awk '{print $2, $1}'
```

Expected output for the exact sample supplied in the question

```
Requests per status code:
200 4
302 1
403 1
404 1
Busiest client:
10.0.0.5 4
Total bytes sent: 43200
```

A separate script is included for analysing a real Nginx combined access log. Standard Nginx combined format uses status field 9 and bytes field 10, unlike the simplified assignment sample. The two field layouts must not be mixed.

Disk-space alert and cron

The disk alert script checks filesystem use with `df -P`, compares each percentage against a configurable threshold (80% by default), and appends a timestamped OK or warning record. A cron schedule runs real-log analysis hourly and the disk alert every 15 minutes. Absolute paths must be substituted for the student's actual installation directory.

```
0 * * * * /bin/bash /absolute/path/scripts/log_analysis.sh /var/log/nginx/access.log
*/15 * * * * THRESHOLD=80 /bin/bash /absolute/path/scripts/disk_space_alert.sh
```

4. B1.3 – systemd Service, Nginx and Network Security

A small Python standard-library sample portal is provided with a home page at `/` and a health endpoint at `/healthz`. It binds only to 127.0.0.1:8000. The systemd unit runs it under a dedicated

non-login account named `portal`, enables automatic startup and restarts it after failure. Nginx listens on port 80 and reverse-proxies requests to the loopback application port.

Deployment validation commands

```
sudo systemctl daemon-reload
sudo systemctl enable --now result-portal
sudo systemctl status result-portal --no-pager
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
curl -i http://127.0.0.1/
curl -i http://127.0.0.1/healthz
sudo journalctl -u result-portal -n 30 --no-pager
```

UFW is configured with default-deny inbound policy and explicit allowances for SSH (22/tcp), HTTP (80/tcp) and HTTPS (443/tcp). If SSH uses a different port, that port must be allowed instead. When configuring a remote server, verify the SSH rule before enabling UFW and retain an active recovery session to avoid locking out the administrator.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

SSH key authentication is established by generating an Ed25519 key on the client and installing only its public key on the server. The private key must remain secret. Password authentication should only be disabled after key login has been tested and recovery access is available.

5. B1.4 – Comparison with Part A Design

Design area	Implemented approach	Evaluation
Service isolation	Dedicated `portal` system account	Limits the service's privileges.
Service management	systemd with restart-on-failure	Supports boot startup and recovery after process failure.
Web entry point	Nginx reverse proxy to loopback port 8000	Keeps the application listener off the public interface.
Network controls	UFW allows SSH, HTTP and HTTPS	Reduces exposed inbound services.
Monitoring	Command collector and service logs	Provides repeatable operational evidence.
Automation	Cron-based log and disk checks	Reduces manual repetitive checks.

The implementation uses a lightweight Python HTTP server as a laboratory substitute for the production portal. It demonstrates process management, loopback binding, reverse proxying and health checks, but it is not a production application server. A production deployment should use a hardened WSGI/ASGI server, TLS certificates, secure secret management, tested backups and monitoring. Any other difference from the student's Part A proposal should be recorded based on the actual design and environment.

6. Validation and Evidence

The deliverables include the completed sample script, real-log analyser, disk alert, system observation collector, sample portal, systemd unit, Nginx site configuration and a measurement worksheet. The student should run these on the Linux VM/server and retain the actual output files.

Evidence to capture

1. Timestamped system observation output for B1.1.
2. Exact output of the completed sample-log script.
3. Real Nginx log-analysis output after generating HTTP requests.
4. Disk alert output and `crontab -l`.
5. systemd status, Nginx syntax test, curl health checks and UFW status.
6. Completed measurement worksheet with explanations.
7. Part A comparison noting every implementation change.

7. Conclusion

The B1 implementation provides a repeatable Linux administration workflow: observe the host, automate recurring checks, manage a sample service through systemd, expose it through Nginx, and restrict inbound access with a firewall. Completion must be supported by command outputs and measurements collected from the student's own Linux environment. The production recommendation is to retain the same operational separation while replacing the laboratory web server with a hardened application stack.